

Máster en Inteligencia Artificial
para Liderazgo Técnico y Transformación Empresarial

Observabilidad y Monitorización LLMOps

Módulo 8 · Ingeniería de Modelos y LLMOps

Tracing, evaluación continua, drift detection, LangSmith, Langfuse y el dashboard mínimo de producción.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Observabilidad y Monitorización LLMOps

Un sistema LLM sin observabilidad es un sistema que falla en silencio. La degradación de la calidad de las respuestas puede ocurrir gradualmente a lo largo de semanas sin ningún error técnico visible: el modelo base de un proveedor cambia de versión, la distribución de las preguntas de los usuarios evoluciona, el corpus RAG se desactualiza, o el prompt deja de funcionar bien para nuevos tipos de consultas. Sin un sistema de monitorización que detecte estas degradaciones, los equipos se enteran del problema cuando los usuarios se quejan.

La observabilidad en sistemas LLM tiene dimensiones que no existen en sistemas de software tradicionales: la calidad del output (que no se puede evaluar solo con métricas técnicas como errores HTTP), el razonamiento del modelo (que puede capturarse con tracing detallado de cada paso del pipeline), y el drift semántico de los inputs (que requiere análisis de embeddings y distribuciones, no solo de estadísticas de latencia).

Las dimensiones de la observabilidad LLM

Tracing: el log detallado de cada pipeline

El tracing captura cada paso del pipeline LLM como un "span": la consulta del usuario, el proceso de retrieval (documentos recuperados, scores de relevancia), la construcción del prompt (contenido exacto enviado al LLM), la llamada al LLM (modelo, temperatura, tokens de entrada/salida, latencia), y el post-procesamiento (extracción de citas, formateo). Los traces permiten reproducir exactamente cualquier interacción pasada y diagnosticar dónde falló un pipeline específico.

Evaluación continua: medir la calidad a escala

La evaluación continua muestrea regularmente las interacciones del sistema y las evalúa con métricas automáticas o LLM-as-judge. Los enfoques más comunes: evaluación offline de una muestra diaria del 5-10% de las interacciones con métricas de calidad (faithfulness, answer relevancy, calidad LLM-as-judge), feedback explícito del usuario (thumbs up/down, ratings), y detección automática de patrones de fallo (respuestas muy cortas, respuestas con "no puedo responder eso", alta varianza en respuestas a preguntas similares).

Drift detection: detectar cuándo la distribución cambia

El drift detection monitoriza si la distribución de los inputs del sistema está cambiando de formas que pueden afectar la calidad. Las técnicas incluyen: embedding drift (comparar la distribución de embeddings de las consultas actuales vs el baseline), topic drift (detectar si están emergiendo nuevos temas que el sistema no manejaba antes), y performance drift (comparar las métricas de calidad de la semana actual vs la semana anterior).

Implementación práctica de observabilidad

Integración con LangSmith

LangSmith de LangChain requiere una sola variable de entorno para activarse: `LANGCHAIN_TRACING_V2=true`, `LANGCHAIN_API_KEY=tu_api_key`. Una vez activado, todos los runs de LangChain y LangGraph se tracean automáticamente en el dashboard de LangSmith con inputs, outputs, latencia, tokens, coste, y pasos intermedios del chain o agente. Para código no-LangChain, el SDK de LangSmith permite instrumentar manualmente.

Evaluación con LLM-as-judge en escala

Para evaluar la calidad de los outputs en producción sin evaluación humana exhaustiva: configura un evaluador LLM (GPT-4o-mini como juez es un buen balance calidad/coste) con un prompt de evaluación específico para tu caso de uso. Ejecuta el evaluador sobre una muestra diaria del 5-10% de las interacciones. Almacena los scores en una serie temporal. Define alertas cuando el score medio cae más de 2 puntos porcentuales respecto a la semana anterior o cuando el score cae por debajo del umbral de calidad mínima aceptable.

Métricas técnicas y de negocio

Las métricas técnicas que monitorizar: latencia (TTFT, end-to-end, P50/P95/P99), tokens por consulta (input/output separados), coste por consulta, tasa de errores, throughput. Las métricas de calidad: score LLM-as-judge, faithfulness (sistemas RAG), tasa de admisión de ignorancia, tasa de feedback positivo del usuario. Las métricas de negocio: tasa de resolución de tickets (para soporte), tasa de conversión (para e-commerce), tiempo ahorrado por usuario, NPS del sistema.

SECCIÓN 4 · EJEMPLOS REALES

- LangSmith en producción (empresa de software): el equipo activó LangSmith con una variable de entorno. En la primera semana detectaron que el 12% de los queries a un agente de código fallaban silenciosamente (el agente terminaba el loop sin producir código válido). Sin el tracing, estos fallos eran invisibles porque el sistema devolvía una respuesta "válida" que en realidad era el mensaje de error del agente. Con el trace detallado, identificaron que el tool use tenía un parsing error en ciertos tipos de respuestas de función.
- Langfuse auto-hosted en empresa con datos sensibles (banco): no podían usar SaaS por restricciones de compliance. Langfuse desplegado en Kubernetes interno captura todos los traces con datos de clientes (encriptados). El equipo tiene visibilidad completa del comportamiento del sistema sin enviar datos fuera del perimeter de seguridad del banco.
- W&B; Weave para correlación ML + LLM (empresa de investigación): el mismo equipo entrena modelos ML con W&B; y despliega sistemas LLM. Weave integra el monitoreo del LLM en producción con el registro de experimentos de fine-tuning en W&B;, permitiendo correlacionar métricas de entrenamiento con métricas de producción en una sola plataforma.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Capturar solo inputs y outputs finales, sin los pasos intermedios. Para un sistema RAG, el output final correcto puede venir de un retrieval incorrecto que "funcionó por suerte". Sin tracing de los documentos recuperados, los scores de retrieval, y el contenido exacto del prompt, el diagnóstico de problemas es imposible.

Error 2: No guardar las trazas suficiente tiempo. Las degradaciones de calidad pueden ser lentas y solo visibles comparando con datos de hace 2-4 semanas. Con retención de solo 7 días de trazas, el diagnóstico de estas degradaciones graduales es imposible. Define una política de retención de mínimo 30 días, con 90 días para sistemas críticos.

Error 3: Alertas demasiado sensibles producen alert fatigue. Si cada pequeña variación en las métricas dispara una alerta, el equipo dejará de responder a ellas. Define umbrales de alerta basados en la variabilidad natural del sistema (media $\pm$ 2 desviaciones estándar del baseline) para alertas significativas.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El dashboard mínimo de observabilidad LLM para producción: panel técnico (latencia P50/P99, tokens/día, coste/día, tasa de errores), panel de calidad (score LLM-as-judge promedio, % feedback positivo, top-10 errores del periodo), panel de negocio (tasa de resolución, NPS del sistema, casos escalados a humano). Actualización diaria para el panel de calidad, tiempo real para el panel técnico.

El proceso de respuesta a degradación detectada: alerta disparada → revisar el trace de los casos de peor puntuación → identificar patrón de fallo → hipótesis de causa → implementar corrección en staging → evaluar en golden dataset → desplegar gradualmente → monitorizar si el score recupera. Todo el proceso debe estar documentado en un runbook para que cualquier miembro del equipo pueda ejecutarlo.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M8-T1 (Ciclo de vida LLM): la observabilidad es el habilitador de la monitorización y mejora continua en el ciclo de vida.
- M7-T7 (Evaluación RAG): RAGAS en producción es la herramienta de evaluación continua para los sistemas RAG.
- M6-T8 (Evaluación de prompts): la observabilidad cierra el loop entre evaluación offline y monitorización en producción.

SECCIÓN 8 · RESUMEN EJECUTIVO

La observabilidad LLM tiene tres dimensiones: tracing (capturar cada paso del pipeline con inputs, outputs, latencia, tokens), evaluación continua (muestra 5-10% diaria con LLM-as-judge), y drift detection (cambios en distribución de inputs o calidad). Plataformas: LangSmith (mejor integración LangChain), Langfuse (open source, auto-hosting), W&B; Weave (integración ML+LLM), Braintrust (evaluación sistemática), Helicone (proxy ligero). Dashboard mínimo: panel técnico (latencia, tokens, coste), panel de calidad (LLM-as-judge, feedback), panel de negocio (resolución, escaladas). Retención de trazas: mínimo 30 días, 90 para sistemas críticos.